

Memoria virtual y algoritmos de reemplazo de páginas

Memoria virtual

Introducción

Las diferentes técnicas de administración de memoria cumplen con una condición esencial, que es que todo el conjunto de instrucciones de un proceso que se está ejecutando debe estar completamente cargado en la memoria principal, siendo la única alternativa para no tener que cumplir esta condición el uso de superposiciones. Esto



fuera a que el espacio completo de direcciones lógicas debe estar dentro de la memoria principal. Esta condición limita el tamaño de los programas, de manera que éstos no pueden exceder del tamaño de la propia memoria principal, aunque en muchos casos, no es necesario tener el programa completo para poderlo ejecutar.

La memoria virtual plantea la separación de la memoria lógica respecto de la memoria física con lo cual se les proporciona a los procesos una memoria lógica extremadamente grande, cuando sólo está disponible una memoria física de menor tamaño. Aunque los beneficios de la memoria virtual son atractivos se podría plantear el interrogante de si es verdaderamente práctica. La experiencia ha demostrado, sin lugar a dudas, que la memoria virtual realmente funciona. La memoria virtual, basada en paginación o paginación más segmentación, se ha convertido, en la actualidad, en una componente esencial de todos los sistemas operativos contemporáneos.

En un breve listado se podrían destacar lo más relevante de esta técnica de gestión de la memoria:

- La memoria virtual permite iniciar la ejecución de un proceso sin estar cargado en memoria completamente.
- El espacio de direcciones del proceso puede ser mayor que la memoria real.
- El tamaño máximo de un programa no está limitado por la memoria real.
- Aunque ambas requerían del esfuerzo del programador, las técnicas que más se parecían a la memoria virtual eran la asignación por superposiciones o la carga dinámica.
- Las instrucciones que se están ejecutando deben estar en memoria física.
- El hecho de poder ejecutar un programa que no esté totalmente en memoria física presenta las siguientes ventajas:
 - Reduce el uso de memoria.
 - Permite la conexión de más usuarios.
- Si una página no está en memoria en el momento que se la referencia se la carga en memoria.
- La memoria virtual permite a los programadores ver una memoria grande a pesar de ser pequeña.
- La memoria virtual independiza al programador del tamaño de la memoria.
- La implementación de la memoria virtual puede ser por:
 - Paginación por demanda: consiste en cargar las páginas lógicas únicamente cuando son referenciadas mientras tanto la página residirá en memoria auxiliar (dispositivo de intercambio).
 - Paginación adelantada o prepaginación o *prepaging*: plantea traer a la memoria las páginas contiguas a la que se está cargando según el principio de proximidad o localidad espacial.
 - Segmentación por demanda: se basa en cargar en memoria principal los segmentos de un proceso sólo cuando son referenciados.
 - Segmentación paginada: en una combinación de las técnicas de segmentación por demanda y paginación por demanda.

Paginación por demanda

Estructura de una fila en la tabla de páginas

Tomando como punto de partida la paginación simple, su versión virtual se implementa agregando a cada entrada en la tabla de páginas un bit, llamado bit de validez, que indica

si la página se encuentra en el espacio de direcciones lógicas de memoria o no. Pero es mejor ver con más detalle la distribución completa de una de sus filas. La estructura de cada fila depende en gran medida de la arquitectura de la computadora, aunque el tipo de información que se mostrará en la siguiente lista es casi el mismo en todas las computadoras:

- Número de marco de página.
- Bit de presente / ausente o bit de validez. Usualmente si es 1, la página virtual asociada a la dirección lógica está en memoria principal y se puede utilizar, pero, si es 0, la página virtual solicitada no se encuentra actualmente en la memoria principal. El acceso a una página marcada como inválida, es decir que no está cargada en memoria principal, genera una interrupción por fallo de página.
- Los bits de protección indican qué tipo de acceso está permitido. En su forma más simple, este campo contiene 1 bit, con 0 para lectura / escritura y 1 para sólo lectura. Un arreglo más sofisticado es tener 3 bits: uno para habilitar la lectura, otro para la escritura y el tercero para ejecutar la página.
- El bit de modificación, o bit sucio, es un control sobre el uso de la página, modificándose cuando se escribe una página. El bit de modificación será útil cuando el sistema operativo seleccione el marco de página correspondiente para realizar un reemplazo. Cuando la página fue modificada, antes de ser reemplazada, será escrita de vuelta en el disco. En cambio, si no ha sido modificada, se la reemplaza inmediatamente.
- El bit de referencia se modifica toda vez que una página es referenciada, sin importar si es para leer o escribir en ella. Ayuda al sistema operativo a elegir una página para quitarla de memoria principal cuando ocurre un fallo de página.
- El bit de uso de caché permite deshabilitar el uso de la caché para la página.

TLB

Sea el sistema de paginación que sea existen dos cuestiones muy importantes que tratar y son:

- La asociación de una dirección virtual a una dirección física debe ser rápida.
- Si el espacio de direcciones virtuales es grande, la tabla de páginas será grande.

Lo primero resulta como consecuencia de que la vinculación entre direcciones virtuales y físicas debe realizarse con cada referencia de memoria. Como absolutamente todas las instrucciones vienen de la memoria y muchas de ellas hacen referencias a operandos en

memoria también resulta que es necesario hacer una, dos o más referencias a la tabla de páginas por cada instrucción. Luego la búsqueda en la tabla de páginas debe realizarse en el menor tiempo posible, es decir, debe durar mucho menos que el tiempo que demanda la ejecución de una instrucción para evitar que la vinculación se convierta en un cuello de botella importante.

La solución pensada se basó en diseñar para los procesadores un pequeño dispositivo de hardware capaz de asociar direcciones virtuales con direcciones físicas sin pasar por la tabla de páginas. El dispositivo, llamado TLB (*Translation Lookaside Buffer*: Búfer de traducción anticipada de direcciones), se encuentra dentro de la MMU y consiste en un pequeño número de líneas, entre 8 y 64, donde cada línea contiene información acerca de una página, como su número de página virtual, su bit de modificación, su bit de validez, su marco de página físico, etc., todos ellos con una correspondencia uno a uno con los campos en la tabla de páginas.

Aciertos y fallos

Cuando la MMU recibe una dirección virtual para que sea traducida, el firmware se fija primero si el número de página virtual está presente en el TLB. Si la encuentra, y el tipo de acceso no contraría los bits de protección, entonces el marco de página se obtiene directamente del TLB, sin ir a la tabla de páginas. Pero si el número de página virtual está presente en el TLB y la instrucción está tratando de escribir en una página de sólo lectura, se genera un fallo de protección. ¿Qué pasa si el número de página virtual no está en el TLB? En ese caso, la MMU detecta el hecho y hace una búsqueda en la tabla de páginas. Luego realiza el reemplazo en una de las líneas del TLB para actualizarla con la nueva información. Así, si esa página se utiliza al poco tiempo otra vez, se producirá un acierto en el TLB en vez de un fallo.

En la actualidad, si no se da una coincidencia en el TLB, en lugar de que la MMU busque en las tablas de páginas para obtener la referencia a la página que se necesita, sólo ocurrirá un fallo del TLB y se transferirá el problema al sistema operativo. Luego, el sistema debe buscar la página, reemplazar una línea del TLB con la nueva información y continuar con la instrucción que originó el fallo. Como los fallos del TLB ocurren con mucha mayor frecuencia que los fallos de página todo el trabajo debe realizarse en unos pocos ciclos del procesador.

Siempre que se utiliza la administración del TLB mediante software se pueden presentar dos tipos de fallos: uno suave y otro duro. Un fallo suave se da cuando la página

referenciada no está en el TLB, pero si en memoria principal. Por el contrario, un fallo duro ocurre cuando la misma página no está ni en el TLB ni en la memoria principal y resulta necesario acceder al disco para traer la página.

En pocas palabras, cada fallo de página hace que se produzca la siguiente secuencia:

1. Interrupción al sistema operativo.
2. Guardar los registros de usuario y el estado del proceso.
3. Determinar que la interrupción se debe a un fallo de página.
4. Comprobar que la referencia de página era legal y determinar la ubicación de la página en el disco.
5. Ordenar una lectura desde el disco para cargar la página en un marco libre:
 - 5.1. Esperar en la cola correspondiente a este dispositivo hasta que se dé servicio a la solicitud de lectura.
 - 5.2. Esperar el tiempo de búsqueda y/o latencia del dispositivo.
 - 5.3. Comenzar la transferencia de la página hacia un marco libre.
6. Mientras se está esperando, se asignará la CPU a algún otro usuario (planificación de la CPU, que será opcional).
7. Recibir una interrupción del subsistema de E/S de disco (E/S completada).
8. Guardar los registros y el estado del proceso para el otro usuario (si se ejecuta el paso 6).
9. Determinar que la interrupción corresponde al disco.
10. Corregir la tabla de páginas y otras tablas para mostrar que la página deseada se encuentra ahora en memoria.
11. Esperar a que se vuelva a asignar la CPU a este proceso.
12. Restaurar los registros de usuario, el estado del proceso y la nueva tabla de páginas y reanudar la ejecución de la instrucción interrumpida.

Rendimiento de la paginación bajo demanda

El tiempo que demanda un fallo de página está compuesto por:

- Tiempo para atender la interrupción del fallo de página.
- Tiempo para traer la página a memoria.
- Tiempo para reiniciar el proceso.

Y puede afectar significativamente al rendimiento de un sistema informático.

Pero además se deben considerar los siguientes tiempos:

- Tiempo de Acceso Efectivo (TAE): es el tiempo que se tarda en recuperar una página de memoria esté presente en ella o no.
- Tiempo de Acceso a Memoria (TAM): es el tiempo en que se tarda en recuperar una página presente en memoria central.
- Tiempo de Acceso a Disco (TAD): es el tiempo en que se tarda recuperar una página desde disco a memoria principal.

Por lo tanto, será $TAE = TAM + TAD$, pudiéndose comprobar entonces que el TAE resulta directamente proporcional a la frecuencia de fallos de página, de manera que siempre es conveniente implementar distintas formas de disminuir el TAE como:

- Tener en disco un espacio especial, para intercambio donde se accede más rápido.
- Disminuir la frecuencia de fallos de página (menos de 1 cada 2.500.000 accesos).
- Aunque se cargan desde archivos, a medida que se reemplazan se las envía a la zona especial de intercambio en disco.

Tipos de estructuras de tablas de páginas

Respecto de cómo está estructurada la tabla de páginas, las opciones planteadas son réplicas de las utilizadas en paginación sin memoria virtual con la diferencia de que la mayoría de los sistemas operativos almacenan las tablas de páginas también en la memoria virtual en lugar de hacerlo en la real. Estas tablas serán paginadas de la misma forma que lo es un proceso y generarán también fallos de página toda vez que una determinada página de la tabla no está presente en la memoria física y deba ser cargada en ella. Teniendo en cuenta esto es posible entonces sólo recordar cuáles eran las formas empleadas en la paginación de la memoria real y mantener como explicación de su funcionamiento la desarrollada oportunamente. Las tres estructuras planteadas eran:

- Tabla de páginas jerárquica o tabla de páginas multinivel.
- Tabla hash de páginas.
- Tabla de páginas invertidas.

Breve comparación: paginación bajo demanda vs. prepaginación

La política de recuperación es aquella que determina cuándo una página se trae a la memoria principal y son dos las variantes más habituales: la paginación bajo demanda y la paginación adelantada (*prepaging*). Como se ha dicho la paginación bajo demanda establece que una página se trae a memoria sólo cuando se hace referencia a una posición

en dicha página. Si la totalidad de los elementos en la política de administración de memoria funcionan bien, sucederá que cuando un proceso arranca, va a haber una seguidilla de fallos de página. A medida que se traen más y más páginas a la memoria, el principio de proximidad establece que las futuras referencias hechas por el proceso se encontrarán en las páginas recientemente traídas. De esta forma, después cierto tiempo, la situación se estabilizará y el número de fallos de página decrecerá sustancialmente.

Por su parte la paginación adelantada o *prepaging*, plantea traer a la memoria real también otras páginas, distintas de la que causó el fallo de página. Como la paginación adelantada tiene en cuenta las características que tienen la mayoría de los dispositivos de memoria secundaria, tales como los discos, que tienen tiempos de búsqueda y latencia de rotación, si las páginas de un proceso se encuentran almacenadas en la memoria secundaria de forma contigua, resulta mucho más eficiente traer a la memoria real un número de páginas contiguas de una sola vez, en lugar de traerlas una por una a lo largo de un periodo de tiempo mucho mayor. Obviamente, esta política es ineficiente si la mayoría de las páginas que se han traído no se referencian más tarde.

La política de prepaginación tiene un buen rendimiento cuando el proceso comienza o cada vez que ocurre un fallo de página, en cuyo caso el programador tiene que indicar de alguna forma las páginas más necesarias. Este último caso, el del fallo de página, es el más apropiado porque resulta completamente invisible al programador.

No se debe confundir la paginación adelantada con el *swapping* ya que cuando un proceso es sacado de la memoria se le coloca en estado suspendido y todas sus páginas residentes son expulsadas de la memoria. Luego, cuando el proceso es reactivado, todas las páginas que estaban previamente en la memoria principal retornaran a ella.

Algoritmos de reemplazo de páginas

Estrategias de gestión de la memoria virtual

Todo sistema de gestión de la memoria virtual que está basado en la paginación tiene fundamentalmente un par de estrategias que determinan el funcionamiento de todo el sistema de memoria y son las siguientes:

- **Estrategias de reemplazo:** la función de estas estrategias es determinar cuál es la página que debe ser desplazada de la memoria principal para dejar su lugar a la página que será cargada.

- **Estrategias de asignación del espacio en memoria principal a los procesos:** tienen como objetivo decidir cómo se reparte la memoria real entre los distintos procesos en ejecución que existen en un momento determinado.

Ambas estrategias están muy relacionadas entre sí y son fundamentales en la performance del sistema de memoria virtual.

Estrategias de reemplazo

Es posible clasificarlas en dos clases:

1. Estrategias de reemplazo global.
2. Estrategias de reemplazo local.

Las primeras pueden seleccionar, con el fin de resolver un fallo de página de un proceso, un marco que actualmente tenga asociada una página perteneciente a otro proceso. Es decir que, un proceso se puede apropiar de un marco de página que está asignado a otro proceso. En cambio, las segundas necesitan que, para resolver el fallo de página de un proceso, sólo se puede seleccionar un marco de página libre o un marco de página que ya pertenezca al proceso que generó el fallo de página.

Algoritmos de reemplazo de páginas

Primeramente, cabe la siguiente aclaración: todos los algoritmos que a lo largo del tiempo han sido desarrollados pueden utilizarse tanto con estrategias globales como con estrategias locales. Por este motivo cualquier descripción de uno de los algoritmos se planteará sin diferenciar el tipo de estrategia de reemplazo. Hay que tener en cuenta además que, el objetivo primordial de cualquier algoritmo de reemplazo es minimizar la tasa de fallos de página sin que sea necesaria y obligatoria la implementación de un hardware especial. Teniendo esto presente se pueden listar los algoritmos más comunes:

- Algoritmo de reemplazo de páginas óptimo.
- Algoritmo FIFO (*First-In, First-Out*: primera en entrar, primera en salir).
- Algoritmo de reemplazo de páginas de segunda oportunidad o algoritmo del reloj.
- Algoritmo de reemplazo de páginas no usadas recientemente (NRU: *Not Recently Used*) o algoritmo de segunda oportunidad mejorado.
- Algoritmo de reemplazo de páginas menos usadas recientemente (LRU: *Least Recently Used*).

- Algoritmo de reemplazo de páginas no utilizadas frecuentemente (NFU: *Not Frequently Used*).
- Algoritmo de reemplazo de páginas mediante aproximación LRU.

Algoritmo de reemplazo de páginas óptimo

Este algoritmo imposible de implementar plantea que cuando se produce un fallo de página el sistema operativo debe seleccionar para el reemplazo a la página que con toda seguridad no será referenciada próximamente. Como el sistema operativo no tiene manera de saber cuándo será referenciada cada una de las páginas en memoria resulta imposible su implementación.

Algoritmo FIFO (*First-In, First-Out*: primera en entrar, primera en salir)

Consiste en desalojar de la memoria principal a la página que haya permanecido cargada en memoria más tiempo. Aunque su implementación es simple y fácil de realizar, tiene la desventaja de que su rendimiento es bajo y que con determinadas secuencias de referencias desarrolla una forma de actuar muy especial conocida como anomalía de Belady. Esta anomalía consiste en el hecho de que cuantos más marcos de página haya en el sistema la cantidad de fallos de página no disminuye, sino que aumenta.

Como ejemplo de este algoritmo si se tiene una memoria con tres *frames* y la siguiente serie de páginas referenciadas: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1, 2, 7, entonces su aplicación sería la siguiente:

Página solicitada	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	2	7
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7	2	2
Frame 2	-	0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0	0	7
Frame 3	-	-	1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1	1	1
Fallo de página	*	*	*	*		*	*	*	*	*	*			*	*			*	*	*	*	*

Algoritmo de reemplazo de páginas de segunda oportunidad o algoritmo del reloj

Trabaja sobre la base del FIFO, pero con la siguiente variación, cuando se produce un fallo de página y se debe realizar un reemplazo, se controla el bit de referencia de la página más antigua, si este bit es 0 significa que la página no solo es antigua y sino que tampoco se la ha utilizado y por lo tanto se la reemplaza. Pero si el bit de referencia es 1 significa que se la usó, entonces se le da otra oportunidad y se cambia su bit de referencia a 0.

Una forma de implementar este algoritmo es con una cola circular y por cómo trabaja se la llama política del reloj debido a que se pueden mostrar las páginas como si estuviesen distribuidas a lo largo de un círculo. En esta implementación, además del bit de referencia, se usa un puntero para señalar cuál es la próxima página que se sustituirá justo después del marco que acaba de actualizarse. El mencionado bit será 1 cuando una página se carga por primera vez a la memoria o toda vez que una página vuelve a utilizarse. Cuando se debe reemplazar una página, el sistema operativo recorre la cola circular para encontrar una página con su bit de referencia en 0. Cuando encuentra una página con el bit de referencia en 1, reinicia ese bit en 0 y continúa. Si alguna de las páginas de la cola circular tiene el bit de referencia en 0 cuando comienza este proceso, la primera de estas páginas que sea encontrada será seleccionada para ser reemplazada. En el caso de que todas las páginas tengan su bit de referencia en 1, el puntero va a completar un ciclo completo a lo largo de la cola circular imitando el comportamiento de un reloj donde el puntero que recorre la cola se comporta como la aguja del reloj, asignándole un 0 a cada uno de los bits de referencia, deteniéndose en su posición original y reemplazando la página apuntada. Este algoritmo es similar a FIFO (si todas las páginas tienen su bit de referencia en 1, el algoritmo es FIFO puro), salvo que, en la estrategia del reloj, el algoritmo pasará por alto toda página cuyo bit de referencia sea 1.

Como ejemplo de este algoritmo si se tiene una memoria con tres *frames* y la siguiente serie de páginas referenciadas: 0, 2, 1, 4, 2, 3, 2, 4, 5, 6, 5, 4, entonces su aplicación sería la siguiente:

Demanda	-	0	2	1	4	2	3	2	4	5	6	5	4
Frame 1	-	0	0	0	4	4	4	4	4	4	4	4	4
Frame 2	-	-	2	2	2	2	2	2	2	5	5	5	5
Frame 3	-	-	-	1	1	1	3	3	3	3	6	6	6
Bit de referencia Frame 1	0	1	1	1	1	1	1	1	1	0	0	0	1
Bit de referencia Frame 2	0	0	1	1	0	1	1	1	1	1	1	1	1
Bit de referencia Frame 3	0	0	0	1	0	0	1	1	1	0	1	1	1
Fallo de página	-	*	*	*	*		*			*	*		
Frame seleccionado	-	1	2	3	1	2	3	2	1	2	3	2	1
Puntero a frame	1	2	3	1	2	3	1	3	2	3	1	3	2

Algoritmo de reemplazo de páginas no usadas recientemente (NRU: *Not Recently Used*) o algoritmo de segunda oportunidad mejorado

Utiliza el mismo principio del algoritmo de segunda oportunidad, pero ahora utiliza dos bits uno de referencia (indica si se usó o no) y uno de modificación (indica si se modificó o no) separando las páginas en cuatro categorías:

- Clase 0: no ha sido referenciada, no ha sido modificada.
- Clase 1: no ha sido referenciada, ha sido modificada.
- Clase 2: ha sido referenciada, no ha sido modificada.
- Clase 3: ha sido referenciada, ha sido modificada.

En el momento de efectuar el reemplazo, tiene en cuenta el siguiente criterio de eliminación:

- Primero elimina a la página que tiene los dos bits en 0.
- De no existir ninguna página en la situación anterior, reemplaza a la que tiene en cero el de modificación.

Algoritmo de reemplazo de páginas menos usadas recientemente (LRU: *Least Recently Used*)

El algoritmo de reemplazo LRU relaciona cada página con el momento correspondiente al último uso de dicha página. Cuando se debe seleccionar una página para efectuar un reemplazo, el algoritmo LRU selecciona la página que no haya sido utilizada durante más tiempo. Sus implementaciones son:

Mediante una pila

La implementación se basa en mantener una pila de números de página, cuando se hace referencia a una página se la saca de la pila y se la coloca arriba de la misma. De este modo la de arriba es la más recientemente usada mientras que la de abajo es la que hace más tiempo no se utiliza.

Por ejemplo, si se considera la siguiente secuencia de referencias a páginas lógicas: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1, 2, 7, el algoritmo se desarrollaría de la siguiente manera:

Página solicitada	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	2	7
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	0	0	0	0	7

Frame 2		-	0	0	0	0	0	0	0	0	3	3	3	3	3	2	2	2	7	7	7	2	2
Frame 3		-	-	1	1	1	3	3	3	2	2	2	2	2	1	1	1	1	1	1	1	1	1
Pila	Tope	7	0	1	2	0	3	0	4	2	3	0	3	0	1	2	0	1	7	0	1	2	7
	:	-	7	0	1	2	0	3	0	4	2	3	0	3	0	1	2	0	1	7	0	1	2
	Fondo	-	-	7	0	1	2	2	3	0	4	2	2	2	3	0	1	2	0	1	7	0	1
Fallo de página		*	*	*	*		*		*	*	*	*			*	*			*			*	*

Mediante el uso de contadores

En esta implementación se agrega a la tabla de páginas un campo que indica momento de ocurrencia, que es un número asignado secuencialmente de manera creciente.

Mediante una matriz

Esta implementación requiere, para una memoria con n marcos de página, una matriz de n x n bits inicializados todos en 0. Siempre que se referencia la página k se ponen primero todos los bits de la fila k en 1 y luego todos los bits de la columna k en 0. En cualquier momento, la fila cuyo valor binario sea menor es la que menos se usó menos recientemente. En base a los valores binarios es posible establecer un orden de uso creciente.

Algoritmo de reemplazo de páginas no utilizadas frecuentemente (NFU: *Not Frequently Used*)

Este algoritmo utiliza un acumulador por página. Inicialmente vale cero y cada vez que sucede una interrupción del reloj, el sistema operativo explora todas las páginas en memoria y a cada acumulador le suma el bit de referencia que vale 0 o 1. Estos acumuladores cuentan la frecuencia con que se hace referencia a cada página. Cuando ocurre un fallo de página, se selecciona la página con el acumulador más chico para sustituirla.

Algoritmo de reemplazo de páginas mediante aproximación LRU

Como algunos equipos no tienen el soporte de hardware necesario para el LRU, se diseñó para ellos el algoritmo de aproximación LRU. Este consiste en agregar en la tabla de páginas un bit de referencia que indica si se usó o no. Este algoritmo se puede optimizar agregando un byte de referencia en lugar de un bit y de esta manera permite analizar los últimos 8 estados.

Buffering de páginas

Esta estrategia implementa la idea de mantener cierta cantidad de marcos de página disponibles. Al ocurrir un fallo de página, se usa un marco libre sin necesidad de aplicar el algoritmo de reemplazo y, por lo tanto, apropiarse de un marco de página sin liberar otro. ¿Cuándo aplicará el sistema operativo el algoritmo de reemplazo? Cuando detecte que el número de marcos decrece por debajo de un cierto nivel. Entonces aplica el algoritmo tantas veces como sea necesario para obtener que el número de marcos libres sea suficiente. Todas las páginas liberadas que no están modificadas pasarán a la lista de marcos libres, mientras que las que han sido modificadas pasarán a la lista de modificadas. Sin importar en cuál de las listas está una determinada página, la misma puede recuperarse si vuelve a ser referenciada. Además, una vez que una página de la lista de modificadas ha sido copiada a la memoria secundaria, se la pasa a la lista de marcos libres.

Política de asignación de espacio a los procesos

En la actualidad existen dos tipos de políticas de asignación de espacio:

- Política de asignación fija.
- Política de asignación variable o dinámica.

La primera le asigna durante la carga inicial de cada proceso, un número fijo de marcos de memoria principal disponibles para su ejecución. Este número máximo se determina basándose en el tipo de proceso o en datos brindados por el programador o el administrador del sistema. Luego, cuando el proceso genera un fallo de página, para el reemplazo sólo se tendrán en consideración los marcos ya asignados al proceso.

La segunda política permite que la cantidad de marcos asignados a un proceso varíe durante la existencia del proceso. De acuerdo con esta política si un proceso está generando una tasa de fallos de página alta de forma continua, se le otorgarán marcos de página adicionales para reducir esta tasa de fallos. Por el contrario, si un proceso tiene una tasa de fallos muy baja, se le disminuirá la cantidad de marcos asignados, esperando que al hacerlo no se incremente notoriamente la tasa de fallos.

Planteadas estas políticas y teniendo presente las estrategias de reemplazo, se tienen las siguientes tres combinaciones más habituales:

- Asignación fija, ámbito local.
- Asignación variable, ámbito global.

- Asignación variable, ámbito local.

Hiperpaginación

Cuando el número de marcos de página asignados a un proceso no es suficiente para almacenar todas las páginas referenciadas por el proceso, ocurrirá que el número de fallos de página aumentará de manera significativa. A este evento que posee una tasa de intercambio de páginas extremadamente alta recibe el nombre de hiperpaginación o *thrashing*. Cuando esto le ocurre a uno o a varios procesos, los mismos estarán más tiempo bloqueados que en ejecución.

Al presentarse la hiperpaginación se debe suspender uno o varios procesos para dejar libres las páginas que tenían asignadas y resulta imprescindible fijar una estrategia de control de carga que ajuste el grado de multiprogramación en el sistema para evitar que la hiperpaginación se vuelva a producir. Algunas políticas de control de carga son las siguientes:

- Estrategia del conjunto de trabajo.
- Estrategia de administración basada en la frecuencia de fallos de página.
- Estrategia de control de carga para algoritmos de reemplazo globales.

La primera de ellas se basa en establecer los conjuntos de trabajo de todos los procesos activos para intentar mantenerlos residentes en memoria principal. Se suele definir al conjunto de trabajo de la siguiente forma: El conjunto de páginas que utiliza un proceso en un momento dado se conoce como su conjunto de trabajo. Cuando todo el conjunto de trabajo está en memoria principal el proceso se ejecutará sin tener muchos fallos de página. Cuando la memoria real disponible es muy chica como para almacenar a todo el conjunto de trabajo entonces el proceso tendrá muchos fallos de página generando el hiperpaginado. Si el sistema operativo tiene la capacidad de establecer cuál es el conjunto de trabajo de un proceso, se puede implementar una estrategia de asignación dinámica con reemplazo local y control de carga de la siguiente forma:

- Cuando el conjunto de trabajo de un proceso disminuye entonces se liberan los marcos asignados a aquellas páginas que ya no están en el conjunto de trabajo.
- Cuando el conjunto de trabajo de un proceso aumenta entonces se le asignarán marcos de página para que puedan cargarse las nuevas páginas que formarán parte del conjunto de trabajo. En el caso de que no haya marcos disponibles, habrá

que realizar un control de carga suspendiendo a uno o a varios de los procesos para disponer de sus páginas.

La segunda estrategia quiere controlar la frecuencia de fallos de página de cada proceso, para lo cual utiliza el algoritmo PFF (*Page Fault Frequency*: Frecuencia de fallo de páginas) que indica cuándo se debe agrandar o reducir la cantidad de páginas asignadas a un proceso sin decir nada acerca de cuál es la página que se debe sustituir al ocurrir un fallo. Como para muchos de los algoritmos de reemplazo de páginas que existen, es sabido que la tasa de fallos decrece cuando se asignan más páginas, el algoritmo PFF utiliza este planteo para desarrollar su trabajo, manteniendo la tasa de paginación para cada proceso dentro de un límite superior y otro inferior.

La última de las estrategias se plantea reducir el número de procesos que compiten por la memoria intercambiando algunos de ellos y así liberar todas las páginas que tenían asignadas para dárselas a otros procesos que están hiperpaginando con el fin de que el mismo se detenga. Si el hiperpaginado no disminuye, la estrategia plantea la repetición del intercambio hasta que se detenga el hiperpaginado.

Ahora bien, se debe considerar otro condicionante: el grado de multiprogramación. Si la cantidad de procesos en la memoria principal es muy baja, el procesador puede entrar en una inactividad prolongada, por lo cual resulta conveniente considerar no sólo el tamaño del proceso y su tasa de paginación, cuando se debe decidir qué proceso se intercambiará, sino también las características del proceso bajo análisis.

Paginación y segmentación combinadas

Básicamente, un sistema combinado de paginación / segmentación, tiene el espacio de direcciones del proceso dividido en un número de segmentos, según el diseño del programador. Luego, cada segmento será dividido en una cantidad de páginas. Si un segmento tiene longitud inferior a una página, el segmento ocupará únicamente una página. Según la visión del programador, toda dirección lógica continuará siendo un número de segmento y un desplazamiento dentro de dicho segmento, pero, según la visión del sistema, el desplazamiento dentro del segmento estará compuesto por un número de página y un desplazamiento dentro de la página incluida en el segmento. Algunos sistemas tienen la capacidad de, mediante la aplicación de las características del hardware del procesador y del sistema operativo, combinar las ventajas de ambos esquemas de administración de memoria.

Bibliografía consultada

- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F. y Pérez Costoya, F. (2001). *Sistemas Operativos: Una visión aplicada* (2ª ed.). Buenos Aires: Mc Graw Hill.
- Silberschatz, A. y Galvin, P. B. (2006). *Fundamentos de Sistemas Operativos* (7ª ed.). México: Addison Wesley Logman.
- Stallings, W. (2005). *Sistemas operativos: Principios de diseño e interioridades* (5ª ed.). Madrid: Pearson/Prentice Hall.
- Tanenbaum, A. S. y Van Steen, M. (2009). *Sistemas operativos modernos* (3ª ed.). México: Pearson/Prentice Hall.